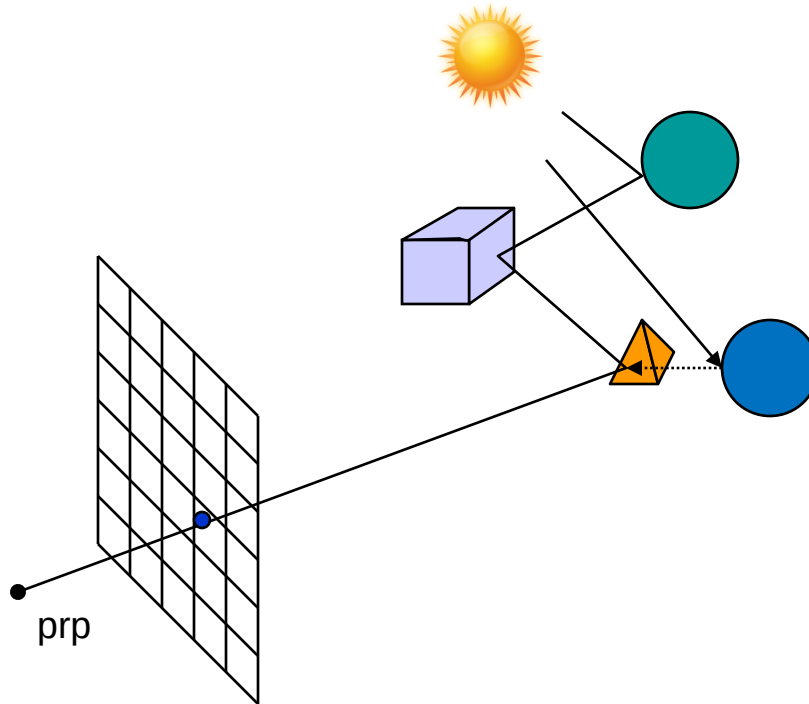


Ray Tracing

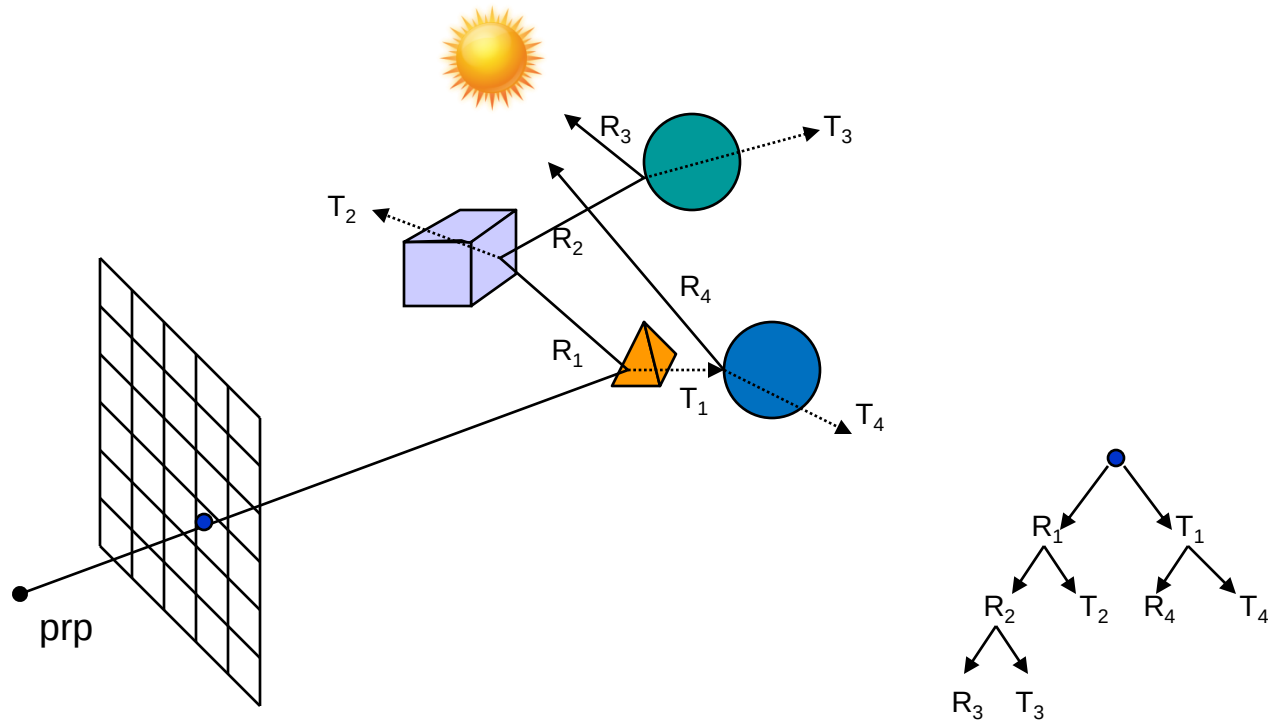
Ray Tracing

- Highly realistic
- Considerable computation time

Ray Tracing



Ray Tracing



Ray Tracing Algorithm

```
main()
{
    Select prp

    for (each scan line)
        for (each pixel in scan line)
            determine ray from prp through pixel
            pixel = Trace (ray, 1)
}
```

```
Trace (ray, depth)
{
    determine closest intersection of ray
    with an object

    if (object hit)
        compute normal at intersection
        return Shade( closest object hit, ray,
                      intersection, normal, depth)
    else
        return background_value
}
```

```
Shade ( object, ray, point, normal, depth)
{
    color = ambient term

    for (each light)
        Sray = ray to light from point
        if (N.L>0)
            compute how much light is blocked by opaque and
            transparent surfaces,
            use it to scale diffuse and specular terms and
            add term to color

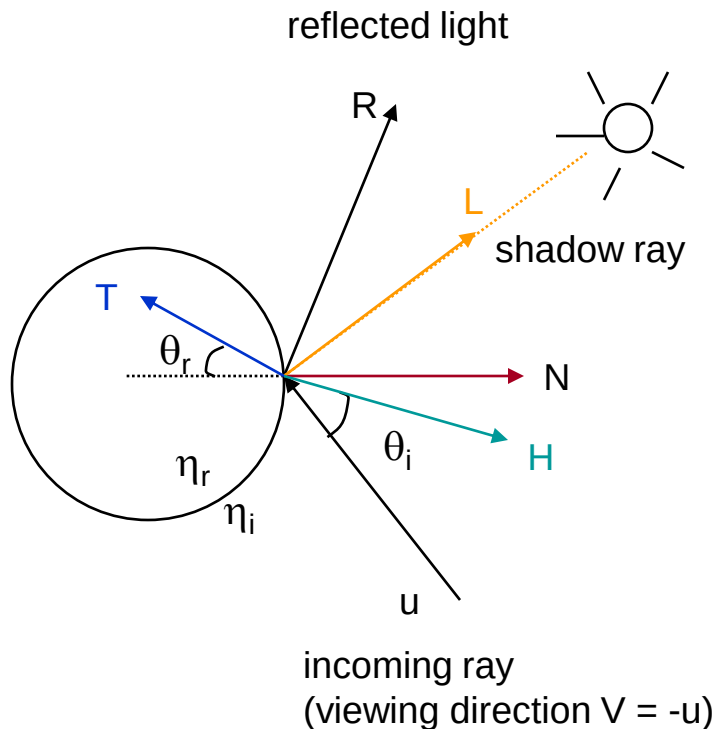
    if (depth < maxdepth)

        if (object is reflective)
            Rray = ray in reflection direction from point
            Rcolor = Trace (Rray, depth+1)
            scale Rcolor by specular coeff. and add to color

        if (object is transparent)
            Tray = ray in refraction direction from point
            if (total internal reflection does not occur)
                Tcolor = Trace (Tray, depth+1)
                scale Tcolor by transmission coeff. and
                add to color

    return color
}
```

Ray Tracing



ambient light: $k_a I_a$

diffuse light: $k_d (N.L)$

specular light: $k_s (H.N)^{ns}$

specular reflection direction:

$$R = u - (2u.N)N$$

$$T = [(\eta_i/\eta_r) \cos \theta_i - \cos \theta_r] N - (\eta_i/\eta_r)L$$

$$\cos \theta_r = \sqrt{1 - (\eta_i/\eta_r)^2 (1 - \cos^2 \theta_i)}$$

Ray Tracing

Ray equation

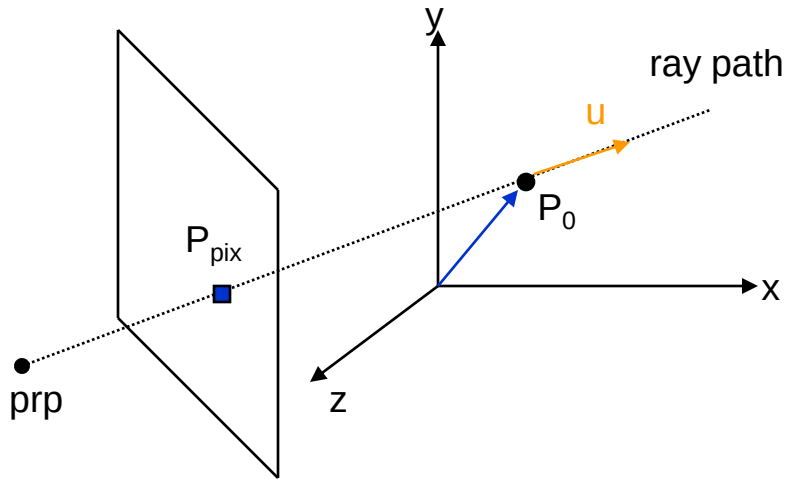
$$\mathbf{P} = \mathbf{P}_0 + s \cdot \mathbf{u}$$

$\mathbf{P}$: any point along the ray

$\mathbf{P}_0$: initial position vector

s : distance of $\mathbf{P}$ to $\mathbf{P}_0$

$\mathbf{u}$: unit direction vector

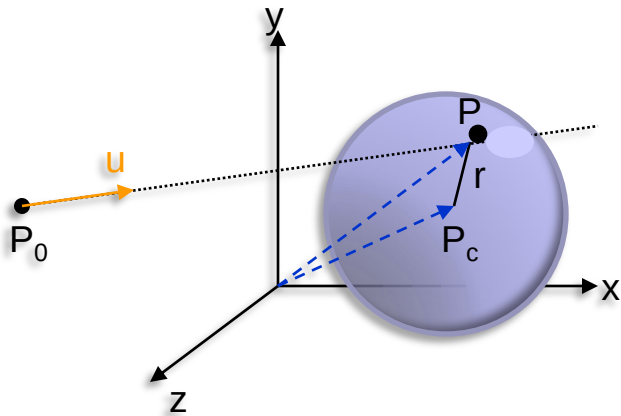


$$\mathbf{u} = \frac{\mathbf{P}_{pix} - \mathbf{P}_{prp}}{|\mathbf{P}_{pix} - \mathbf{P}_{prp}|}$$

Initially $\mathbf{P}_0$ is $\mathbf{P}_{pix}$ or $\mathbf{prp}$.

Update $\mathbf{P}_0$ and $\mathbf{u}$ at each intersection point on the ray with a surface.

Ray Tracing



P : intersection point of the sphere along the ray

P_c : center of sphere

r : radius of sphere

$$|P - P_c|^2 - r^2 = 0$$

$$|P_0 + s \cdot u - P_c|^2 - r^2 = 0$$

$$\rightarrow s = u \cdot \Delta P \pm \sqrt{(u \cdot \Delta P)^2 - |\Delta P|^2 + r^2}$$

where $\Delta P = P_c - P_0$

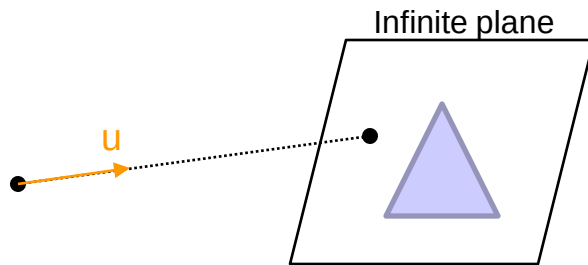
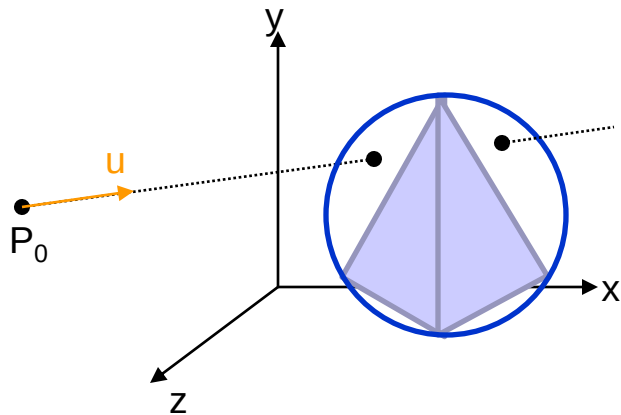
If discriminant < 0

ray does not intersect sphere

Otherwise

calculate P using $P = P_0 + s \cdot u$

Ray Tracing



If the ray does not intersect the sphere,
eliminate the polyhedra

Otherwise
do the following:

Identify front faces using: $u \cdot N < 0$

For each face that satisfies $u \cdot N < 0$:

- Solve plane equation:

$$N \cdot P = -D$$

$$N \cdot (P_0 + s \cdot u) = -D$$

$$\rightarrow s = - \frac{(D + N \cdot P)}{N \cdot u}$$

- Perform **inside-outside test** to check
if the intersection is inside the polygon

Ray Tracing

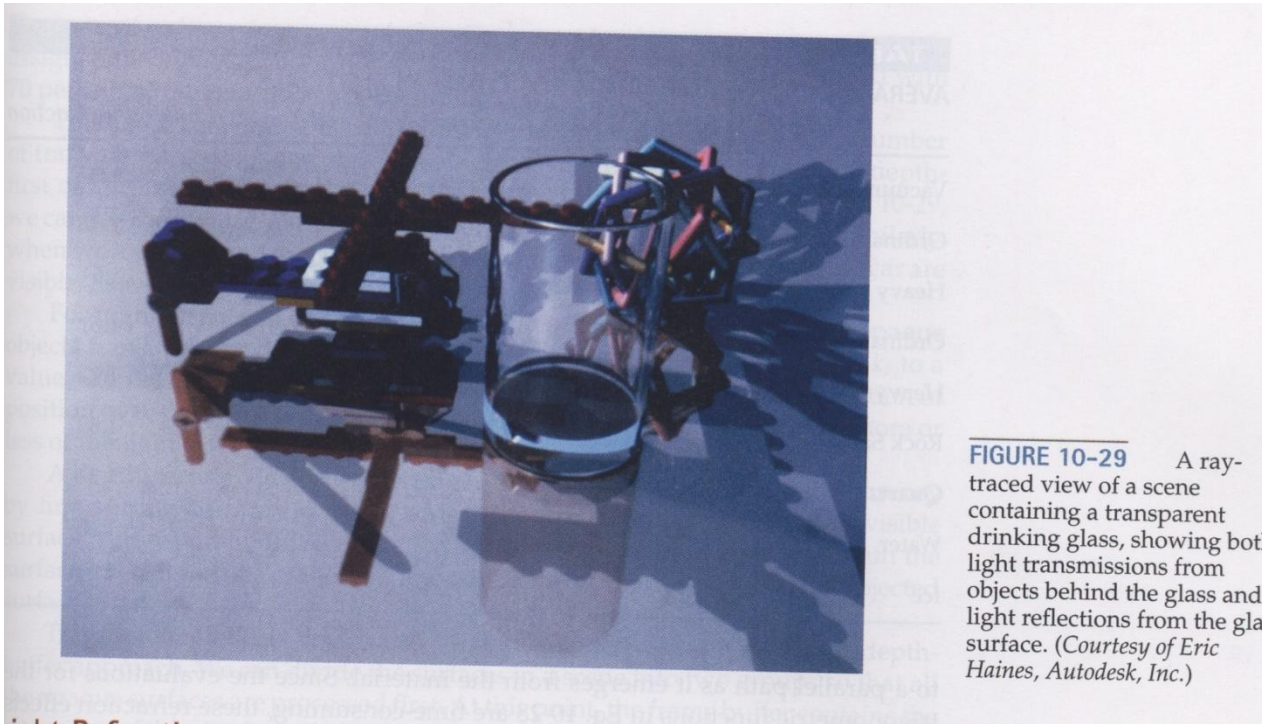


FIGURE 10-29 A ray-traced view of a scene containing a transparent drinking glass, showing both light transmissions from objects behind the glass and light reflections from the glass surface. (Courtesy of Eric Haines, Autodesk, Inc.)

Ray Tracing

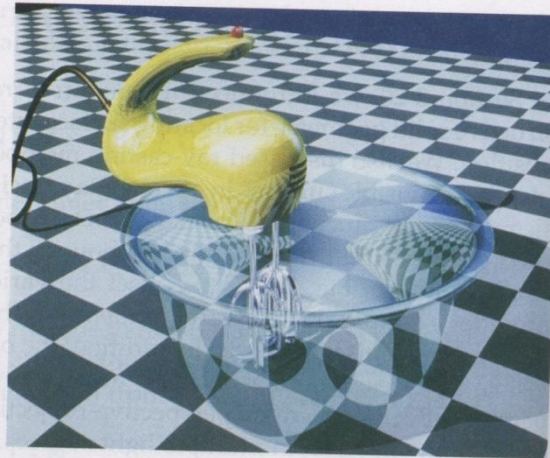


FIGURE 10-53 A ray-traced scene, showing global reflection and transparency effects.
(Courtesy of Evans & Sutherland.)

Ray Tracing



Ray Tracing

